ETSI TS 104 286 V1.1.1 (2026-08)



TECHNICAL SPECIFICATION

Methods for Testing & Specification (MTS); Methodology Definition; Digital Transformation of Security Standards into Requirements

Reference
DTS/MTS-TST12TraStaReq

Keywords
IoT, LLM, methodology, security

ETSI

650 Route des Lucioles
F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - APE 7112B
Association à but non lucratif enregistrée à la
Sous-Préfecture de Grasse (06) N° w061004871

Important notice

The present document can be downloaded from the
ETSI Search & Browse Standards application.

The present document may be made available in electronic versions and/or in print. The content of any electronic and/or print versions of the present document shall not be modified without the prior written authorization of ETSI. In case of any existing or perceived difference in contents between such versions and/or in print, the prevailing version of an ETSI deliverable is the one made publicly available in PDF format on ETSI deliver repository.

Users should be aware that the present document may be revised or have its status changed, this information is available in the Milestones listing.

If you find errors in the present document, please send your comments to the relevant service listed under Committee Support Staff.

If you find a security vulnerability in the present document, please report it through our Coordinated Vulnerability Disclosure (CVD) program.

Notice of disclaimer & limitation of liability

The information provided in the present deliverable is directed solely to professionals who have the appropriate degree of experience to understand and interpret its content in accordance with generally accepted engineering or other professional standard and applicable regulations.
No recommendation as to products and services or vendors is made or should be implied.
No representation or warranty is made that this deliverable is technically accurate or sufficient or conforms to any law and/or governmental rule and/or regulation and further, no representation or warranty is made of merchantability or fitness for any particular purpose or against infringement of intellectual property rights.
In no event shall ETSI be held liable for loss of profits or any other incidental or consequential damages.

Any software contained in this deliverable is provided "AS IS" with no warranties, express or implied, including but not limited to, the warranties of merchantability, fitness for a particular purpose and non-infringement of intellectual property rights and ETSI shall not be held liable in any event for any damages whatsoever (including, without limitation, damages for loss of profits, business interruption, loss of information, or any other pecuniary loss) arising out of or related to the use of or inability to use the software.

Copyright Notification

No part of this document may be reproduced in any form, by any means and in any media, without the prior written authorization of ETSI and except as expressly permitted below.
By way of exception and when the document is a normative deliverable (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)), ETSI authorizes to reproduce and incorporate into products, services and technical documentation only those extracts (e.g. templates) that are strictly necessary for the technical implementation of the normative deliverable, to ensure compliance with the latter.
Nothing in this notice shall be construed as limiting any mandatory exceptions to copyright provided by applicable law.

© ETSI 2026.
All rights reserved.

Contents

Intellectual Property Rights 4
Foreword 4
Modal verbs terminology 4
Executive summary 4
Introduction 5
1 Scope 7
2 References 7
2.1 Normative references 7
2.2 Informative references 7
3 Definition of terms, symbols and abbreviations 8
3.1 Terms 8
3.2 Symbols 8
3.3 Abbreviations 9
4 Background 9
5 Methodology Overview 9
6 Methodology 10
6.1 Workflow 10
6.2 Adaptation Techniques 11
6.3 Document Parsing (DP) 12
6.4 Requirement Identification and Extraction (RIE) 12
6.4.1 Fine-tuning 12
6.4.1.1 Overview 12
6.4.1.2 Dataset Construction and Data Preparation 13
6.4.1.3 Model Selection 14
6.4.1.4 Model Evaluation 14
6.4.1.5 Model Execution 14
6.4.2 Prompt Engineering 15
6.5 Requirement Classification (RC) 15
6.5.1 Fine-tuning 15
6.5.1.1 Overview 15
6.5.1.2 Dataset Construction and Data Preparation 15
6.5.1.3 Preprocessing and Model Construction 16
6.5.1.4 Model Evaluation 16
6.5.1.5 Model Execution 16
6.5.2 Prompt Engineering 16
6.6 Requirement Checklist Generation (RCG) 17
6.7 Feedback Provision (FP) 17
Annex A (normative): Examples of extracted requirements 19
History 23

Intellectual Property Rights

Essential patents

IPRs essential or potentially essential to normative deliverables (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)) may have been declared to ETSI. The declarations pertaining to these essential IPRs, if any, are publicly available for **ETSI members and non-members**, and can be found in ETSI SR 000 314: *"Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"*, which is available from the ETSI Secretariat. Latest updates are available on the ETSI IPR online database.

Pursuant to the ETSI Directives including the ETSI IPR Policy, no investigation regarding the essentiality of IPRs, including IPR searches, has been carried out by ETSI. No guarantee can be given as to the existence of other IPRs not referenced in ETSI SR 000 314 (or the updates on the ETSI Web server) which are, or may be, or may become, essential to the present document.

Trademarks

The present document may include trademarks and/or tradenames which are asserted and/or registered by their owners. ETSI claims no ownership of these except for any which are indicated as being the property of ETSI, and conveys no right to use or reproduce any trademark and/or tradename. Mention of those trademarks in the present document does not constitute an endorsement by ETSI of products, services or organizations associated with those trademarks.

DECT™, **PLUGTESTS™**, **UMTS™** and the ETSI logo are trademarks of ETSI registered for the benefit of its Members. **3GPP™**, **LTE™** and **5G™** logo are trademarks of ETSI registered for the benefit of its Members and of the 3GPP Organizational Partners. **oneM2M™** logo is a trademark of ETSI registered for the benefit of its Members and of the oneM2M Partners. **GSM**® and the GSM logo are trademarks registered and owned by the GSM Association.

Foreword

This Technical Specification (TS) has been produced by ETSI Technical Committee Methods for Testing and Specification (MTS).

Modal verbs terminology

In the present document "**shall**", "**shall not**", "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the ETSI Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in ETSI deliverables except when used in direct citation.

Executive summary

The present document introduces a methodology for the semi-automated transformation of security standards into a list of security requirements. More specifically, it describes an AI-driven methodology for the identification, extraction, and classification of security requirements from text retrieved from security standards documents. The methodology is based on adapting pre-trained foundational Large Language Models (LLMs) using common adaptation techniques like prompt engineering and fine-tuning in the downstream tasks of security requirements identification, extraction, and classification. The present document can also be used as a guide for practitioners who are interested in utilizing LLMs for automating the identification, extraction, and/or classification of security requirements from security standards.

Introduction

Compliance with international security standards is essential for ensuring the security of digital products and services, and thereby their dependability and trustworthiness. Beyond fostering trust, compliance with security requirements is increasingly mandated by regulatory frameworks in the European Union (EU), such as the Cyber Resilience Act (CRA) [i.1]. In particular, under the CRA, products with digital elements should comply with critical product-level cybersecurity requirements before being placed on the EU market. The CRA entered into force on 10 December 2024**,** and its main obligations apply from 11 December 2027 (with certain obligations applying earlier). Therefore, compliance with international security standards and regulations has become highly relevant to a broad set of stakeholders and application domains, including some that previously had fewer formal cybersecurity obligations.

Compliance evaluation is typically conducted by security experts (e.g. internal assessors, auditors, etc.) who verify whether critical security requirements expressed in relevant international security standards or regulations are satisfied by the subject system. To state it simply, the security expert works from a list of security requirements that the system needs to satisfy and gathers objective evidence (e.g. technical documentation, test results, etc.) regarding the satisfaction of these requirements. If sufficient evidence is found for the considered requirements, the system can be regarded as fully compliant with the designated/selected security standard or regulation.

Compliance evaluation starts with the extraction of the security requirements that are expressed in a relevant standard or regulation that the system needs to comply with, and their representation in the form of a list of clearly-described and well-structured requirements. Extracting requirements from standards is a tedious, time-consuming, and effort-demanding task, which is also prone to human error. This is mainly attributed to the fact that international security standards are often long documents with complex, highly technical language, requiring technical knowledge and expertise by the person who is tasked to go through their text and detect and extract the security requirements. Until today, this process remains largely manual. Although several rule-based and Natural Language Processing (NLP) techniques have been introduced, they have not achieved the accuracy and generalizability required for broader adoption in practice.

However, the recent advancements in the field of Artificial Intelligence (AI) and particularly the proposition of the Transformer architecture and its derivative LLMs that have demonstrated remarkable capabilities in natural language understanding and processing open a new potential for the automation of the digital transformation process of security standards and regulations.

The present document introduces an AI-based methodology for the semi-automated digital transformation of security standard documents into lists of actionable security requirements, which can be used as the foundation for conducting subsequent compliance evaluation activities. The methodology is based on adapting LLMs in order to be able to:

i) identify and extract security requirements that are described in the documents of international security standards and regulations;

ii) categorize/classify them into high-level security criteria/characteristics to which they belong (e.g. Confidentiality, Integrity, Availability, etc.); and

iii) extract the identified requirements in a unified form, which is both machine- and human-readable.

The methodology for the digital transformation of security standards is based on two separate LLM-based mechanisms, one for the identification and extraction of the security requirements from text retrieved from international security standards and regulations, and another for the classification of the extracted requirements into specific security categories. These two mechanisms are used jointly in order to deliver the desired functionality, acting as independent collaborative AI agents, each one excelling in its designated task and acting synergistically in order to achieve the high-level goal of turning a standard document into a well-structured list (checklist) of security requirements.

The proposed methodology is based on the adaptation of pre-trained foundational LLMs, leveraging common adaptation techniques, including prompt engineering (both zero- and few-shot learning) and fine-tuning. The most suitable adaptation technique needs to be selected for each one of the considered tasks, i.e. requirements identification, extraction, and classification. The present document, apart from the methodology itself, provides guidelines on how to adapt existing LLMs in order to be used as the basis for these important requirements engineering tasks.

The methodology is intended for use by conformity assessment bodies and auditors preparing requirement checklists to be used as the basis for subsequent compliance and certification activities, by product and security engineering teams deriving security requirements from applicable standards, by standards owners and working groups piloting semi-automated requirement-extraction workflows, by tool developers and integrators building LLM-assisted compliance tooling, and by regulators and market-surveillance authorities exploring repeatable methods for requirements derivation.

The present document can be used for deriving a clearly structured set of requirement records from a referenced standard or regulation, configuring and adapting LLMs for requirement identification, extraction and classification, establishing traceability from each derived requirement to the source clause for auditability, supporting human-in-the-loop review and quality control, harmonising internal compliance datasets and processes across products and projects, and benchmarking alternative prompts, models and settings against common acceptance criteria.

The present document is independent of any specific product, vendor, model family, deployment environment or programming framework. It does not prescribe a particular LLM, model size, decoding strategy, training corpus, cloud provider or tooling stack, and it is compatible with both proprietary and open-source implementations, whether deployed on-premises or in cloud environments. Any examples or parameter values are illustrative only and do not imply endorsement. References to external documents are non-exhaustive and provided for context.

1 Scope

The present document introduces an AI-driven methodology for enabling the semi-automated transformation of international security standards into a comprehensive list of security requirements. More specifically, the present document specifies a methodology for:

a) The automatic identification and extraction of security requirements from text retrieved from standard documents.

b) The classification of security requirements extracted from standards into critical high-level security categories/attributes (e.g. Confidentiality, Integrity, Availability, etc.).

c) The semi-automated transformation of complete standard documents into a list of well-structured security requirements, incorporating a feedback loop with the user.

d) Adapting LLMs in the tasks of security requirements identification, extraction and classification.

2 References

2.1 Normative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

Referenced documents which are not found to be publicly available in the expected location might be found in the ETSI docbox.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents are necessary for the application of the present document.

Not applicable.

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long-term validity.

The following referenced documents may be useful in implementing an ETSI deliverable or add to the reader's understanding, but are not required for conformance to the present document.

[i.1] Regulation (EU) 2024/2847 of the European Parliament and of the Council of 23 October 2024 on horizontal cybersecurity requirements for products with digital elements and amending Regulations (EU) No 168/2013 and (EU) 2019/1020 and Directive (EU) 2020/1828 (Cyber Resilience Act), OJ L, 20.11.2024.

[i.2] Object Management Group (2016): "Requirements Interchange Format (ReqIF)", Version 1.2. OMG Document Number formal/2016-07-01.

[i.3] ISO/IEC Directives Part 2 (Edition 9) (2021): "ISO/IEC Directives, Part 2: 'Principles and rules for the structure and drafting of ISO and IEC documents", Edition 9, 2021.

[i.4] Gotel OC, Finkelstein CW: "An analysis of the requirements traceability problem". In Proceedings of IEEE international conference on requirements engineering 1994 Apr 18 (pp. 94-101). IEEE.

[i.5] Zhong R, Xu Y, Zhang C, Yu J.: "Leveraging large language model to generate a novel metaheuristic algorithm with CRISPE framework". Cluster Computing. 2024 Dec; 27(10):13835-69.

[i.6] Almeida J. Prompt engineering: "A comparative study of prompting techniques in AI language models". In 2025 IEEE Integrated STEM Education Conference (ISEC) 2025 Mar 15 (pp. 1-4). IEEE.

[i.7] Andress J.: "The basics of information security: understanding the fundamentals of InfoSec in theory and practice". Waltham, MA: Syngress. 2014.

[i.8] NIST SP 800-160 (Volume 1): "Engineering Trustworthy Secure Systems".

[i.9] NIST SP 800-53 (Revision 5) (2020-09): "Security and Privacy Controls for Information Systems and Organizations". National Institute of Standards and Technology.

[i.10] Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence and amending Regulations (EC) No 300/2008, (EU) No 167/2013, (EU) No 168/2013, (EU) 2018/858, (EU) 2018/1139 and (EU) 2019/2144 and Directives 2014/90/EU, (EU) 2016/797 and (EU) 2020/1828 (Artificial Intelligence Act), OJ L, 12.7.2024.

[i.11] ETSI EN 303 645 (V3.1.3) (2024-09): "CYBER; Cyber Security for Consumer Internet of Things: Baseline Requirements".

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the following terms apply:

digital transformation of standards: process of converting unstructured textual content of a standard into structured, traceable, and machine-processable requirement records

hallucination: generation of content by a Large Language Model that is nonsensical, inaccurate, or unfaithful to provided source material, yet presented in a confident and convincing manner

human-in-the-loop: AI development approach that integrates human interaction and expertise into the machine learning cycle

requirement record: structured representation of an extracted requirement including its text, source reference, and assigned security category

requirement statement: textual expression corresponding to a single identifiable security requirement extracted from a standard

security category: predefined classification label representing a high-level security property to which a requirement may belong

security requirement: normative statement within a standard that expresses an obligation, prohibition, or permission related to security properties

traceability: ability to link an extracted requirement record to its original source location within a standard document

textual fragment: logically coherent portion of a standard document (e.g. clause, subclause, paragraph) used as input for requirement identification and extraction

3.2 Symbols

Void.

3.3 Abbreviations

For the purposes of the present document, the following abbreviations apply:

AI	Artificial Intelligence
BART	Bidirectional and Auto-Regressive Transformer
BERT	Bidirectional Encoder Representations from Transformers
CIA	Confidentiality, Integrity, and Availability triad
CRA	Cyber Resilience Act
DP	Document Parsing
EU	European Union
FP	Feedback Provision
GPT	Generative Pre-trained Transformer
IoT	Internet of Things
JSON	JavaScript Object Notation
LLM	Large Language Model
MTS	Methods for Testing and Specification
NLP	Natural Language Processing
RC	Requirements Classification
RCG	Requirement Checklist Generation
RIE	Requirements Identification and Extraction
XMI	eXtensible Markup Language Metadata Interchange
XML	eXtensible Markup Language

4 Background

Requirements are best represented not as isolated text fragments but as uniquely identifiable information objects. Each object should include the normative requirement statement together with the metadata needed to support aspects such as interpretation, provenance, classification, lifecycle management, rationale, verification method, and traceability. Depending on the use case, this metadata can include a unique identifier, source artefact and location, type or classification, version, and explicit relationships to other requirements and/or design elements, verification artefacts, etc. [i.2].

The preservation of structural and contextual information is especially important when requirements are extracted from standards or other normative documents. In such sources, the meaning of a requirement may depend on a lead-in sentence, list structure, table context, or incorporated references. Consequently, requirement extraction has to preserve not only the requirement statement itself, but also the surrounding structural context needed to interpret it correctly and to reconstruct its normative intent [i.3].

Explicit representation of dependencies, references, and trace links is equally important. Traceability supports forward and backward navigation across the lifecycle of a requirement, thereby enabling impact analysis, coverage analysis, change management, and more reliable automation. A requirement representation that omits typed relationships may still be readable to humans, but it is much less effective for rigorous analysis and digital processing [i.4].

Structured and machine-readable representations allow requirements, metadata, and relationships to be exchanged and processed consistently across tools and organizational boundaries [i.2].

5 Methodology Overview

The input for the workflow described in the methodology is the complete document of a security standard (or regulation) or fragments retrieved from the document, which is then analysed by AI models to semi-automatically identify, extract, and classify security requirements that are present in the text of the analysed standard document. The output is a list with the extracted security requirements in a human- and machine-readable format (see clause 6.6).

The workflow of the methodology is organized into three phases: a Parsing Phase, responsible for preparing the input document for analysis; an Intelligence Phase, responsible for identifying, extracting, and classifying security requirements; and a Reporting Phase, responsible for generating the resulting requirement checklist and incorporating user feedback. The high-level overview of the methodology is illustrated in Figure 1.

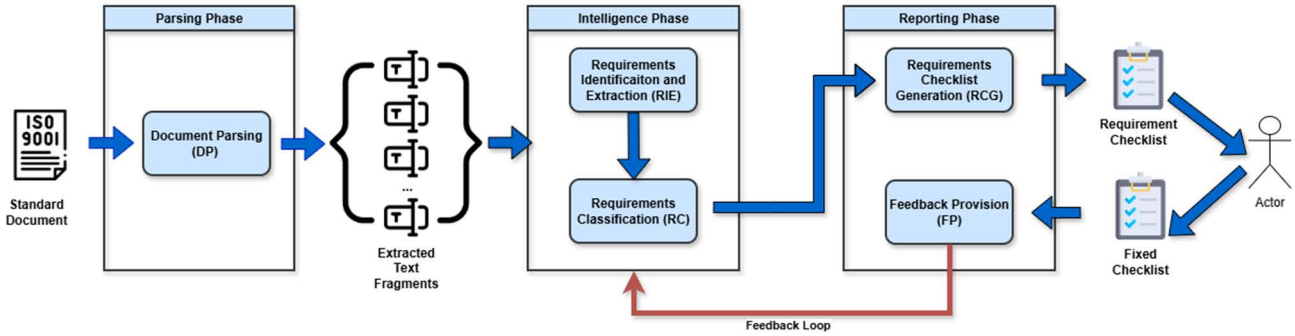


Figure 1: The high-level overview of the proposed methodology for the digital transformation of security standards

Each of the three phases consists of internal steps that collaboratively support the transformation process. The steps in the workflow are described below:

- **Parsing Phase**
  - **Document Parsing (DP):** This step constitutes the entry point of the workflow. It receives a standard or regulation document and prepares its content for subsequent processing. This step produces structured textual fragments together with associated contextual information (i.e. clauses and sections from which the fragments originate) and provides them to the Requirements Identification and Extraction step.
- **Intelligence Phase**
  - **Requirements Identification and Extraction (RIE):** This step receives the textual fragments that are extracted by the DP step and analyses them to identify and extract security requirements. This step produces structured and traceable requirement records containing the extracted requirements and associated information (i.e. requirement identifier, clause and sub-clause in the standard). The output of the RIE step is provided to the Requirements Classification and Requirement Checklist Generation steps.
  - **Requirements Classification (RC):** This step receives the requirements identified by the RIE step in order to classify each requirement to a security category. The output of the RC step consists of a set of classified requirement items that preserve the traceability information provided by the preceding steps and can be further processed by the RCG step.
- **Reporting Phase**
  - **Requirement Checklist Generation (RCG):** This step receives the requirement records produced by the RIE and RC steps and consolidates the extracted requirements, assigned security categories, and associated information into a structured representation. The step produces a security requirements checklist that constitutes the primary output of the digital transformation methodology.
  - **Feedback Provision (FP):** This step enables users to inspect the produced requirements checklist and provide feedback by validating, correcting, rejecting, or supplementing the requirements. The feedback produced by the FP step is provided to the RIE and RC steps to support subsequent refinement of the underlying models and configurations.

6 Methodology

6.1 Workflow

The workflow integrates DP, RIE, RC, RCG and FP steps. RIE and RC constitute the core analytical mechanisms, while DP, FP and RCG steps provide supporting functionality. The requirements for these steps are outlined below:

a) The DP step:

   a) shall act as the entry point of the digital transformation of standards pipeline;

b) shall transform a standard document into structured textual fragments suitable for processing by the RIE step;

c) shall communicate with the RIE step by providing segmented textual fragments (e.g. clauses or paragraphs) along with the clause and page identifiers (see clause 6.3).

b) The RIE step:

a) shall consume the textual fragments produced by the DP step and process them to identify the security requirements contained in them;

b) shall preserve the clause and page identifiers associated with each textual fragment so that every extracted requirement remains traceable to its corresponding location within the source document (see clause 6.4).

c) The RC step:

a) shall process each requirement statement and assign a security category according to the predefined categories list as described in clause 6.5;

b) shall pass the structured requirement records to the RCG and FP steps (see clause 6.5).

d) The RCG step:

a) shall aggregate requirement identifier, extracted text, corresponding clause and page, and assigned security category in one record per requirement and generate machine-readable representations (e.g. JSON, XML);

b) shall not modify semantic content when producing its output (i.e. requirement checklists) (see clause 6.6).

e) The FP step:

a) shall receive the requirements checklist produced by the RCG and shall make it available for review by a human expert;

b) shall enable the human expert to accept, reject, modify, or supplement the generated requirements. This process shall support the correction of incorrectly extracted requirements, refinement of requirement wording, adjustment of assigned security categories, and addition of omitted requirements based on expert's judgement;

c) shall make feedback results available to the RIE and RC steps to be used as new data samples (see clause 6.7).

This bidirectional communication establishes an iterative improvement loop that enhances the accuracy and stability of both RIE and RC over time. It also reduces the risk of providing to the user output affected by LLM hallucinations.

6.2 Adaptation Techniques

The RIE and RC are based on the adaptation of pre-trained LLMs to the corresponding tasks. The adaptation technique shall be selected according to the characteristics of the target application, the availability of labelled data, the computational resources, and the expected performance. The methodology supports two major complementary adaptation techniques:

- **Prompt engineering:** An adaptation technique for enabling the utilization of pre-trained LLMs for designated tasks through prompting, without updating the parameters of the model via further training. Two types of prompt engineering are recommended:

  i) zero-shot learning where the prompt contains only instructions for performing the desired task without providing any examples; and

  ii) few-shot learning where the prompt contains both instructions and indicative examples. More details are provided in clause 6.4.2.

- **Fine-tuning:** An adaptation technique for enabling the utilization of pre-trained LLM for a designated task through updating its parameters via further training on a dataset that is specific to the desired task.

Prompt engineering enables the use of a pre-trained LLM without additional training, although its performance can depend on the capabilities of the selected model and the formulation of the prompts. Fine-tuning enables the model to be adapted to the designated task using task-specific data but requires labelled data and additional training, which can be costly or infeasible due to the need for high-performance computing infrastructure.

Fine-tuning shall be preferred over prompt engineering when:

- smaller models need to be utilized for reducing the cost (e.g. energy consumption) at inference time;
- prompt engineering demonstrated insufficient performance for the desired standards or family of standards, and therefore, the LLMs need to be fine-tuned to further improve their accuracy;
- focus is given on a specific standard or family of standards, and therefore, fine-tuning may be considered a more viable option;
- the organization has the computational power required for fine-tuning LLMs.

The application of LLMs to the processing of standards can be subject to limitations such as the context-window size of the selected model, loss of contextual information resulting from document segmentation, sensitivity to model and prompt configurations, and availability of hardware resources. The methodology covers both adaptation techniques, in order to cover a broad range of scenarios, stakeholders, use cases, and potential limitations.

6.3 Document Parsing (DP)

DP constitutes the first step of the digital transformation methodology and prepares the source document for subsequent requirements identification and extraction. The DP step shall accept as input a security standard or regulation (or part of it) in one of the various formats in which standard documents are commonly distributed (e.g. PDF, DOCX, XMI, etc.). Then, the DP step shall split the document into a set of sequential textual fragments. Textual fragments may correspond to clauses, subclauses, paragraphs, or other logical units of the source document.

The DP step shall pre-process the produced fragments to bring them into a form suitable for being processed by the LLM-based requirements identification, extraction, and classification models. In particular, the relationship between textual content and document structures relevant to the interpretation of requirements, including clause and subclause boundaries, lead-in statements, lists, and associated list items, shall be preserved where present in the source document. Where a textual fragment depends on preceding or subsequent content for its interpretation, sufficient context shall be retained or associated with the fragment to support requirements identification and extraction.

The DP step shall associate each textual fragment with metadata sufficient to identify its location within the source document. Such metadata shall include the document identifier, clause or subclause identifier, and page. The structured textual fragments shall be provided as input to the RIE step.

6.4 Requirement Identification and Extraction (RIE)

6.4.1 Fine-tuning

6.4.1.1 Overview

Requirements identification and extraction can be achieved via fine-tuning of existing pre-trained LLMs. Fine-tuning is based on the precondition that a reliable dataset containing textual fragments from standard documents, along with the extracted requirements is available for use.

The high-level overview of the fine-tuning-based methodology is illustrated in Figure 2.

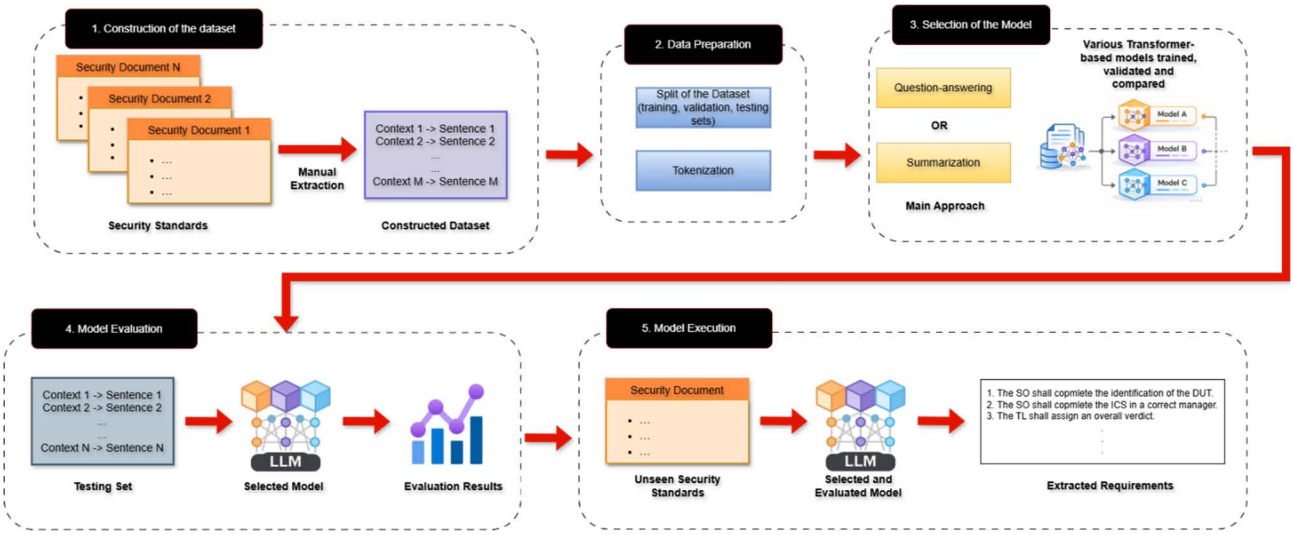


Figure 2: The high-level overview of the fine-tuning process for adapting Large Language Models (LLMs) for security requirements identification and extraction from standards text fragments

The fine-tuning process consists of five different steps:

1) Dataset Construction,
2) Data Preparation,
3) Model Selection,
4) Model Evaluation, and
5) Model Execution.

The first four steps are responsible for building the Requirements Identification and Extraction model, whereas the last step corresponds to the usage of the produced model in practice. The main elements of this process are described in the following clauses.

6.4.1.2 Dataset Construction and Data Preparation

The first step of the methodology is the construction of a domain-specific dataset, which is required for fine-tuning the selected LLMs. The dataset shall consist of pairs of textual fragments (i.e. context) extracted from security standards or regulations and the corresponding security requirement statements (i.e. sentences) contained within those fragments. The textual fragments shall be paragraphs, clauses, or other logically coherent sections of the source standard.

The associated requirement statements shall correspond to the explicit security requirements expressed within the corresponding context. The dataset construction process shall ensure consistency in annotation criteria across all samples, traceability between each requirement statement and its source clause, appropriate coverage of different requirement types and security domains.

The next step of the methodology is data preparation. Data preparation shall prepare the data for the model selection process. The dataset shall be divided into training, validation, and testing sets. The training and validation sets shall be used for fine-tuning the LLMs and selecting the best model, whereas the testing set shall be utilized for evaluating the model's performance on unseen data samples. The data samples shall be tokenized to generate token (i.e. word) sequences, which constitute the required format for the input of the LLMs.

6.4.1.3 Model Selection

The model selection step focuses on adapting pre-trained LLMs to the downstream task of security requirements identification and extraction, using the domain-specific dataset described in clause 6.4.1.2. Model selection shall determine an appropriate model configuration that can accurately identify and extract security requirement statements from textual fragments of security standards. For this purpose, a variety of proprietary and/or open-source pre-trained LLMs can be used, including Transformer-based architectures, such as BERT™, BART™, T5™, RoBERTa™, and DistilBERT™. Pre-trained LLMs shall be fine-tuned using supervised learning techniques.

The expected output shall correspond to the associated security requirement statements. Therefore, it is considered a Sequence-to-Sequence task, but different task formulations can be adopted for this adaptation process.

The pre-trained LLMs shall be fine-tuned on one of two objectives for the specific task of requirements identification and extraction:

- Question-Answering objective;
- Summarization objective.

In Question-Answering, the requirement extraction task shall be structured as a question-answering problem. The model shall:

- receive a textual fragment from a security standard (context);
- receive a predefined query prompt;
- be trained to generate the corresponding requirement statement or statements as the answer.

In Summarization, the requirement extraction task shall be structured as a targeted summarization problem. The model shall:

- receive a textual fragment from a security standard;
- be configured to minimize abstraction;
- preserve the normative meaning of the original text, ensuring that extracted requirements remain semantically aligned with the source content;
- produce a condensed output containing exclusively the security requirement statements present in the fragment.

The selection of the most appropriate formulation and model configuration shall be based on evaluation results obtained using the validation dataset. Performance shall be assessed using quantitative metrics such as ROUGE-L, precision, recall, and F_1-score, as well as qualitative analysis of semantic fidelity and structural correctness. The selected model shall demonstrate at least consistent identification of requirement statements across different clause structures and reservation of the original normative intent.

6.4.1.4 Model Evaluation

During model's evaluation, the model configuration identified as best-performing during the validation process (i.e. model selection step) shall be applied to unseen labelled data samples (i.e. the testing set). The evaluation results shall be used to quantify the model's performance using the selected evaluation metrics. The evaluation results shall be recorded to provide evidence of the performance of the selected model configuration on unseen test data.

6.4.1.5 Model Execution

The purpose of the model execution step is to apply the evaluated model configuration to operational data for the identification and extraction of security requirements. During the model execution step (i.e. inference time), the selected model configuration shall be applied to previously unseen standards or regulations that were not included in the dataset construction, training, validation, or testing processes.

6.4.2 Prompt Engineering

Prompt engineering is an adaptation technique considered suitable for security requirements identification and extraction. Knowledge of the structure and the drafting rules of the analysed standard or family of standards can be incorporated into the prompt to guide the LLM in distinguishing requirement statements from other textual content. Where drafting conventions relevant to the analysed standard are known (e.g. the use of modal verbs and other normative expressions), the prompts shall incorporate applicable information concerning the expression and structure of normative content.

The following example prompts illustrate the effect of incorporating drafting conventions into the prompt:

EXAMPLE 1: "Detect and extract security requirements in the following text fragment: <text_fragment>".

EXAMPLE 2: "Detect and extract security requirements in the following text fragment retrieved from an ETSI standard: <text_fragment>. According to ETSI Drafting Rules requirements in ETSI standards use the words SHALL, SHOULD and MAY. They do not use words like CAN (they are used for informative content)".

In EXAMPLE 1, the model relies on its prior knowledge (what it had learned during its training) to decide what can be considered a security requirement. In EXAMPLE 2, the rules based on which the requirements are structured are explicitly stated, particularly the keywords that would indicate a requirement in the standard document. Provided that the standard document and its associated requirements are written following these rules, the LLM is expected to be more able to identify and extract the requirements that reside in the text, compared to the case of a freely written text without following any particular rules or guidelines. The more detailed the prompt and the better it reflects the drafting rules based on which the selected standards were prepared, the more accurate the identification and extraction result is expected to be [i.5].

Prompt engineering may be applied in zero-shot or few-shot configurations. In zero-shot prompting, only task instructions shall be provided. In few-shot prompting, representative examples of textual fragments and their corresponding extracted requirements shall be included within the prompt to guide the model's behaviour. Few-shot configurations can further improve extraction consistency, particularly when dealing with specialized terminology or domain-specific phrasing [i.6].

The prompt-based method does not require model fine-tuning and may therefore be applied directly to general-purpose LLMs. However, its performance may vary depending on the complexity of the analysed standard, the clarity of the drafting structure, and the selected LLM.

6.5 Requirement Classification (RC)

6.5.1 Fine-tuning

6.5.1.1 Overview

RC can be achieved via fine-tuning of pre-trained LLMs. A labelled dataset containing security requirement statements and their associated security categories shall be available for fine-tuning. Representative security categories include the security objectives of the CIA triad (i.e. Confidentiality, Integrity and Availability) [i.7], [i.8], as well as complementary security properties commonly addressed by international cybersecurity standards, including Authenticity and Privacy [i.9]. The fine-tuning-based classification methodology shall include the steps of dataset preparation, model construction, and evaluation.

6.5.1.2 Dataset Construction and Data Preparation

The dataset used for requirements extraction shall be augmented to support classification by associating each extracted requirement statement with one security category label. Each record in the classification dataset shall consist of a security requirement statement and its corresponding security category label.

The dataset shall be prepared and partitioned in accordance with the general principles specified in clause 6.4.1.2. To preserve representativeness across categories, stratified splitting techniques shall be applied to ensure that each subset retains the original class distribution.

6.5.1.3 Preprocessing and Model Construction

Before training, the dataset shall undergo preprocessing to ensure consistency and compatibility with the selected model architecture. The classification model shall be constructed by adapting a pre-trained Transformer-based architecture and attaching a classification head to its output layer. Suitable model families include but are not limited to GPT™-based models, BART™, T5™, BERT™ and its variants (e.g. DistilBERT™, RoBERTa™). During fine-tuning, the model learns to associate semantic patterns within requirement statements with the predefined security categories.

6.5.1.4 Model Evaluation

The testing set shall be used for the evaluation that takes place to assess the classification performance of the fine-tuned model. Specifically, the testing set shall be used exclusively to evaluate model performance on unseen data in order to estimate generalization capability. Common classification evaluation metrics include accuracy, precision, recall, and F_1-score.

Macro-averaged metrics shall be used when all security categories are considered equally important. Macro-averaging computes the arithmetic mean of the per-class metric values, without weighting them according to class frequency. Among these metrics, the F_1-score is particularly informative, as it represents the harmonic mean of precision and recall. It captures both false positives and false negatives and is therefore suitable for evaluating classification performance in scenarios where class imbalance may occur.

6.5.1.5 Model Execution

The purpose of the model execution step is to apply the evaluated classification model to operational data in order to assign security categories to previously extracted requirements. During the model execution step (i.e. inference time), the constructed security requirements classification model shall be applied to previously unseen security requirement statements that were not included in the dataset construction, training, validation, or testing processes. This step corresponds to the utilization of this tool in practice.

6.5.2 Prompt Engineering

Prompt engineering can also be applied to the task of security requirements classification. Given the semantic reasoning capabilities demonstrated by LLMs, instruction-based classification constitutes a viable alternative to fine-tuning, particularly in scenarios where labelled datasets are limited or unavailable.

Unlike the requirement identification and extraction task (clause 6.4.2), which leverages drafting rules and modal verbs to detect normative statements, the classification task relies primarily on semantic interpretation of the requirement content. Therefore, prompt design focuses on clearly defining the classification taxonomy.

The implementation shall construct prompts that explicitly define the classification taxonomy and guide the model towards consistent category assignment. The implementation shall include clear category definitions and explicit output format instructions within the prompts. For illustration, consider the following example prompts for requirement classification:

EXAMPLE 1: "Classify the following security requirement into one of the following categories: Confidentiality, Integrity, Availability, Authenticity, and Privacy.

Requirement: <requirement_text>"

EXAMPLE 2: "You are a security requirement classifier. Classify each of the given security requirements into exactly ONE of the following categories: Confidentiality, Integrity, Availability, Authenticity, and Privacy. Return ONLY the category in lowercase. Do NOT add punctuation, explanations, or extra text.

Requirement: <requirement_text>"

EXAMPLE 3: "You are a security requirement classifier. Classify each of the given security requirements into one of the following categories: Confidentiality, Integrity, Availability, Authenticity, and Privacy. Return ONLY the category in lowercase. Do NOT add punctuation, explanations, or extra text.

- Confidentiality: protection against unauthorized disclosure of information.
- Integrity: protection against unauthorized modification or corruption of data or systems.

- Availability: ensuring systems, services, and data are accessible and usable when required.
- Authenticity: verification of the identity of users, devices, or data sources.
- Privacy: protection of personal data and compliance with data protection principles.

Requirement: <requirement_text>"

In EXAMPLE 1, the model is instructed to assign a requirement to one of several predefined categories. The classification relies primarily on the model's prior knowledge of security terminology and the semantic interpretation of the requirement text.

In EXAMPLE 2, the prompt introduces additional constraints that aim to increase determinism and output consistency. The model is explicitly instructed to assign exactly one category, select from a fixed and clearly enumerated list (i.e. Confidentiality, Integrity, Availability, Authenticity and Privacy [i.7], [i.8], [i.9]), return the category in lowercase, and avoid additional commentary or formatting. Such constraints reduce variability in the generated output and facilitate automated post-processing.

When classification results are integrated into structured workflows, the prompt shall define a structured output format and the fields, permitted values, and constraints applicable to the generated output. For example, the classification result may be represented as a JSON object containing the requirement identifier and the assigned security category.

EXAMPLE 3 enhances the prompt in EXAMPLE 2 by explicitly defining each security category within the instruction [i.7], [i.8], [i.9]. Providing semantic definitions reduces ambiguity and improves alignment between the model's interpretation and the intended taxonomy. This method is particularly beneficial when category boundaries may overlap or when domain-specific terminology could lead to inconsistent classification.

6.6 Requirement Checklist Generation (RCG)

The RCG step shall receive the requirement records produced by the RIE and RC steps and shall consolidate the extracted requirements, assigned security categories, and associated metadata into a security requirements checklist. The RCG step shall generate one structured record for each extracted requirement. Each record shall include, at a minimum, a requirement identifier, the extracted requirement text, the assigned security category, and source traceability information. Possible format of a requirement record is provided below:

- Identifier (ID) of the requirement
- Standard title
- Text of the requirement
- Category of the requirement
- Clause (and sub-clauses) of the requirements

The RCG step shall generate the security requirements checklist in a structured format suitable for both human interpretation and machine processing. The selected representation shall define a consistent structure for the requirement records and shall enable their automated parsing and exchange with other systems. The checklist may be represented using JSON, XML, or another structured data format that satisfies these requirements. The RCG step shall preserve the semantic content of the requirement records received from the preceding steps when generating the security requirements checklist. The generated security requirements checklist shall be made available to the FP step for human review and feedback.

6.7 Feedback Provision (FP)

The FP step shall enable human review and feedback on the security requirements checklist produced by the RCG step. After receiving the generated checklist, the user shall review the requirement records and shall use the FP step to provide feedback on identified inaccuracies, inconsistencies, or omissions.

The FP step shall enable the human expert to validate the generated requirement records and identify inaccurate, incomplete, or omitted requirements. The human expert shall be able to accept or reject a requirement record, modify the extracted requirement text, adjust the assigned security category, and add requirements omitted during the identification and extraction process.

The FP step shall capture the results of the review in a structured form. For each reviewed requirement record, the feedback information shall preserve the original generated content, the corrections provided by the human expert, and the validation status of the record. Requirements added during the review process shall be associated with source traceability information where such information can be identified by the human expert. The FP step shall produce a validated security requirements checklist that incorporates the corrections and additions resulting from the human review process.

The feedback information shall be stored as validated corrections associated with the corresponding RIE or RC outputs. Corrections to requirements identification and extraction results shall be collected as new labelled samples for the RIE step, while corrections to assigned security categories shall be collected as new labelled samples for the RC step. The FP step shall accumulate the validated corrections until a predefined refinement criterion is satisfied. The refinement criterion shall be defined according to the selected adaptation technique and may include the accumulation of a predefined number of validated corrections or the expiration of a predefined time interval. For fine-tuned configurations, the collected samples shall be incorporated into the corresponding task-specific dataset and used during subsequent fine-tuning activities. For prompt-based configurations, the collected samples shall be analysed during periodic prompt refinement activities and used to revise task instructions, examples, or other prompt elements.

The FP step shall support human oversight of the generated results and the combination of automated processing with human expertise. Human review reduces the risk that inaccurate, incomplete, or unsupported model outputs are propagated as validated requirements. Human oversight is also addressed as an element of trustworthy AI systems in the AI Act [i.10].

Annex A (normative): Examples of extracted requirements

Table A.1: Input: Text fragment from ETSI EN 303 645 [i.11], clause 5.0

Provision 5.0-1 A justification shall be recorded for each recommendation in the present document that is considered to be not applicable for or not fulfilled by the consumer IoT device.

It is recommended that the manufacturer conducts a threat modelling to identify, analyse and mitigate threats to the device.

Table B.1 provides a schema to record these justifications in a structured manner. This is to allow other stakeholders (e.g. assurance assessors, members of the supply chain, security researchers or retailers) to determine whether provisions have been applied correctly and appropriately.

EXAMPLE 1: The manufacturer publishes a completed version of table B.1 alongside the product description on their website.

EXAMPLE 2: The manufacturer completes table B.1 for internal record keeping. Sometime later, an external assurance organization assesses a product against the present document and requests information relating to the product's security design. The manufacturer can easily provide this information as it is contained within table B.1.

Cases where a provision is not applicable or not fulfilled by the consumer IoT device can include:

- when the use case of a device determines constraints in resources that prevent the implementation of certain security measures or certain security measures are not appropriate to the identified risk (security or privacy);
- where the functionality described in the provision is not included (e.g. a device that only presents data without requiring authentication);
- when the use case for the device requires compliance with additional industry or regulatory requirements that preclude compliance with the provision.

EXAMPLE 3: The consumer IoT device and associated service processes payments and is therefore required to comply with the payment card industry specifications and for transaction processing and industry and regulatory requirements for fraud monitoring. Fraud monitoring can conflict with provision 5.11-2 that recommends allowing users to be able to delete all personal data on the associated service, potentially including the records necessary to detect fraudulent activity.

Table A.2: Output: Extracted requirements from ETSI EN 303 645 [i.11], clause 5.0

A justification shall be recorded for each recommendation in the present document that is considered to be not applicable for or not fulfilled by the consumer IoT device.

Table A.3: Input: Text fragment from ETSI EN 303 645 [i.11], clause 5.2

Provision 5.2-1 The manufacturer shall make a vulnerability disclosure policy publicly available. This policy shall include, at a minimum:

- contact information for the reporting of issues; and
- timelines for initial acknowledgement of receipt of a vulnerability report; and
- timelines for when the person who reported the issue will receive status updates until the resolution of the reported issues.

NOTE 2: Different notations are possible to describe time values in the timetable (e.g. "7 days", "quickly", etc.). ETSI TR 103 838 [i.30] contains an example for a vulnerability disclosure policy including timelines for initial acknowledgement of receipt and a proposal for information on timelines for status updates until the resolution of the reported issues. The time needed to find a solution can vary depending e.g. on the criticality of the disclosed vulnerability.

A vulnerability disclosure policy clearly specifies the process through which security researchers and others are able to report issues. Such policy can be updated as necessary to further ensure transparency and clarity in the dealings of the manufacturer with security researchers, and vice versa.

Coordinated Vulnerability Disclosure (CVD) is a set of processes for dealing with disclosures about potential security vulnerabilities and to support the remediation of these vulnerabilities. CVD is standardized by the International Organization for Standardization (ISO) in the ISO/IEC 29147 [i.4] on vulnerability disclosure. Moreover, a guide to CVD is given in ETSI TR 103 838 [i.30]. CVD has been proven to be successful in some large software companies around the world. Multi-Party Coordinated Vulnerability Disclosure (MPCVD) refers to the application of CVD in a multi-stakeholder scenario, for example when products from multiple manufacturers are affected by the same vulnerability. Manufacturers can refer to the guidelines from ISO/IEC TR 5895 [i.33] and FIRST [i.32] to adapt their CVD processes accordingly.

In the IoT industry, CVD is currently not well-established [i.16] as some companies are reticent about dealing with security researchers. Here, CVD provides companies a framework to manage this process. This gives security researchers an avenue to inform companies of security issues, puts companies ahead of the threat of malicious exploitation and gives companies an opportunity to respond to and resolve vulnerabilities in advance of a public disclosure.

Table A.4: Output: Extracted requirements from ETSI EN 303 645 [i.11], clause 5.2

- The manufacturer shall make a vulnerability disclosure policy publicly available.
- This policy shall include, at a minimum: contact information for the reporting of issues; and timelines for initial acknowledgement of receipt of a vulnerability report; and timelines for when the person who reported the issue will receive status updates until the resolution of the reported issues.

Table A.5: Input: Text fragment from ETSI EN 303 645 [i.11], clause 5.3

Provision 5.3-16 The model designation of the consumer IoT device shall be clearly recognizable, either by labelling on the consumer IoT device or via a user interface.

This is often performed by communicating with a consumer IoT device over a logical interface.

EXAMPLE 14: A consumer IoT device has a HTTP (or HTTPS when appropriate) API that reports the model designation (after user authentication).

Knowledge of the specific designation of the consumer IoT device is often required to check the defined support period of software updates or the availability of software updates.

Being able to identify devices can assist with update management. Where devices can be offline, or updates fail, tracking the update status of user devices through the associated services can assist the user in maintaining device health across all the devices they own. It is recommended that the IoT device is to be clearly recognizable by having both a model designation and a unique privacy preserving per device identifier, either by labelling on the device or via a physical and logical interface. A unique privacy preserving per device identifier could be generated by the manufacturer or could be an identifier the end user sets. and may not be accessible to anyone but the user of the device and associated services. To preserve privacy, the availability and usage of the unique identifier needs to prevent remote services unauthorized parties from either identifying or tracking the device or its users. Being able to identify devices can assist with update management.

EXAMPLE 15: A consumer IoT device is identified in a user update dashboard by a user-chosen identifier ("KitchenUnit") that describes the device's location in a way that is unique to the user, although unlikely to be unique globally, and would allow the user to identify the device from the name. This identifier is deleted when the device is reset to factory settings.

EXAMPLE 16: A consumer IoT device is identified to the update service by a serial number set by the manufacturer only. As is the only identifying information that the update service receives, there is no method by which the transfer of the device between users can be tracked or the identity of the user of the device known and tracked. This serial number is not re-used as a device identifier for any other purpose.

Table A.6: Output: Extracted requirements from ETSI EN 303 645 [i.11], clause 5.3

- The model designation of the consumer IoT device shall be clearly recognizable, either by labelling on the consumer IoT device or via a user interface.
- A unique privacy preserving per device identifier may not be accessible to anyone but the user of the device and associated services.

Table A.7: Input: Text fragment from ETSI EN 303 645 [i.11], clause 5.5

Provision 5.5-3 Cryptographic algorithms and primitives should be replaceable.

NOTE 1: This is a building block for "cryptoagility".

NOTE 2: Changing cryptographic algorithms and primitives in configuration is a security relevant change.

For devices that cannot be updated, it is important that the intended lifetime of the device does not exceed the recommended usage lifetime of cryptographic algorithms used by the device (including key sizes).

The concept of cryptoagility may cover different aspects such as hardware resources of the consumer IoT device including cryptographic hardware accelerators, background infrastructures such as Public Key Infrastructures and development or design processes that anticipate the lifetime of cryptographic algorithms and primitives.

Table A.8: Output: Extracted requirements from ETSI EN 303 645 [i.11], clause 5.5

• Cryptographic algorithms and primitives should be replaceable. • The concept of cryptoagility may cover different aspects such as hardware resources of the consumer IoT device including cryptographic hardware accelerators, background infrastructures such as Public Key Infrastructures and development or design processes that anticipate the lifetime of cryptographic algorithms and primitives.

History

Version	Date	Status
V1.1.1	August 2026	Publication